```
from google.colab import drive
drive.mount('/content/drive')
```

Show hidden output

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
```

```
df=pd.read_csv("/content/drive/MyDrive/Salary_dataset.csv")
df.head()
df.drop(columns=['Unnamed: 0'])
df.shape
```

```
(30, 3)
```

```
y=np.array(df['Salary']).reshape(-1, 1)
x=np.array(df['YearsExperience']).reshape(-1, 1)
```

```
model=LinearRegression()
```

```
model.fit(x,y)
```

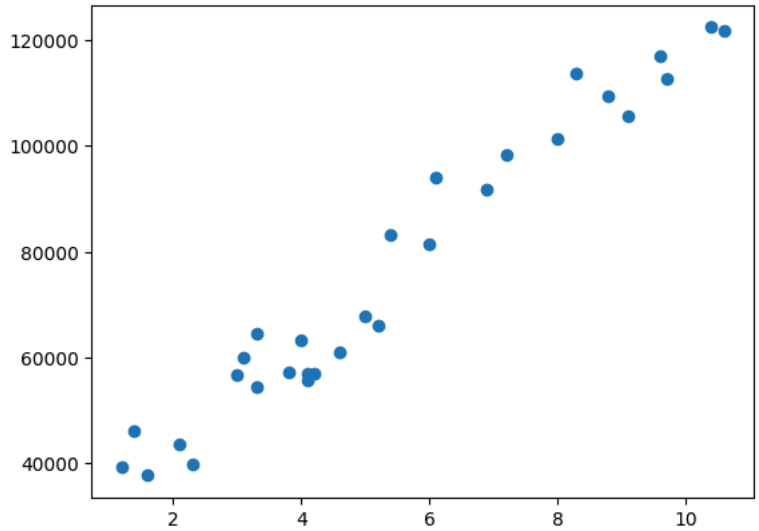
```
▾ LinearRegression
LinearRegression()
```

```
Salary=model.predict([[2.3]])
Salary
```

```
array([[46583.11730587]])
```

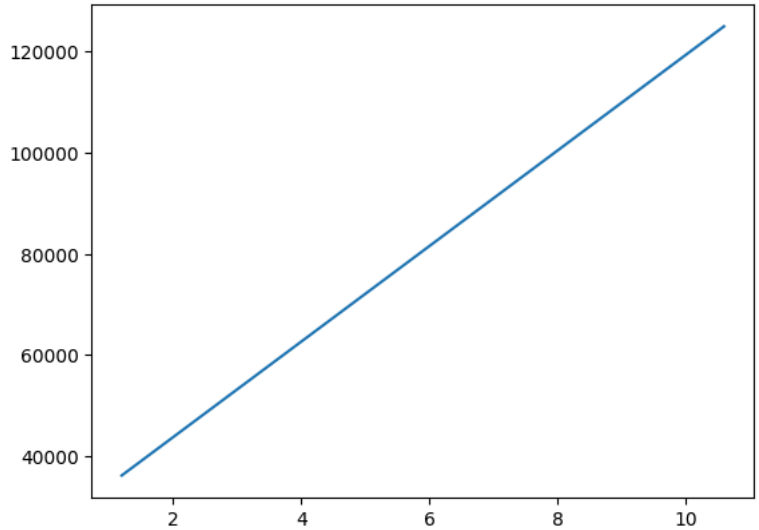
```
plt.scatter(x,y)
```

```
<matplotlib.collections.PathCollection at 0x7b19f30fb2c0>
```



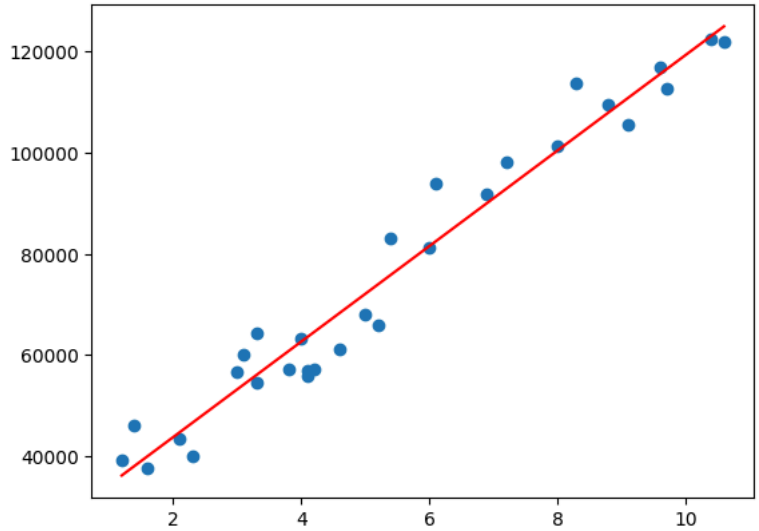

```
y_predict=model.predict(x) #y=mx+c
plt.plot(x,y_predict)
```

```
[<matplotlib.lines.Line2D at 0x7b19f318d370>]
```




```
plt.scatter(x,y)
plt.plot(x,y_predict,c="red")
```

```
[<matplotlib.lines.Line2D at 0x7b19f301edb0>]
```



```
from google.colab import drive
drive.mount('/content/drive')
```

Show hidden output

```
# Choose the path to your CSV in Drive
CSV_PATH = "/content/drive/MyDrive/airline_passengers.csv"
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

```
df = pd.read_csv(CSV_PATH)
print("First 5 rows:")
display(df.head())
```

```
First 5 rows:
```

	Month	Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121

```
data = df["Passengers"].values.reshape(-1,1)
plt.figure(figsize=(10,4))
plt.plot(data, label='Passengers')
plt.title("Monthly Airline Passengers")
plt.xlabel("Time (months)")
plt.ylabel("Passengers")
plt.legend()
plt.show()
```

```
# 2. Normalize data
scaler = MinMaxScaler(feature_range=(0,1))
scaled = scaler.fit_transform(data)
```

```
# 3. Create sequences (look_back = 10 months)
def create_dataset(series, look_back=10):
    X, y = [], []
    for i in range(len(series)-look_back):
        X.append(series[i:i+look_back, 0])
        y.append(series[i+look_back, 0])
    return np.array(X), np.array(y)

look_back = 10
X, y = create_dataset(scaled, look_back)
X = X.reshape((X.shape[0], look_back, 1))  # (samples, timesteps, features)
```

```
# 4. Split train/test
split = int(len(X)*0.67)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

```
# 5. Build simple Vanilla RNN
model = keras.Sequential([
    layers.SimpleRNN(16, input_shape=(look_back,1), activation="tanh"),
    layers.Dense(1)
])
model.compile(optimizer="adam", loss="mse")
```

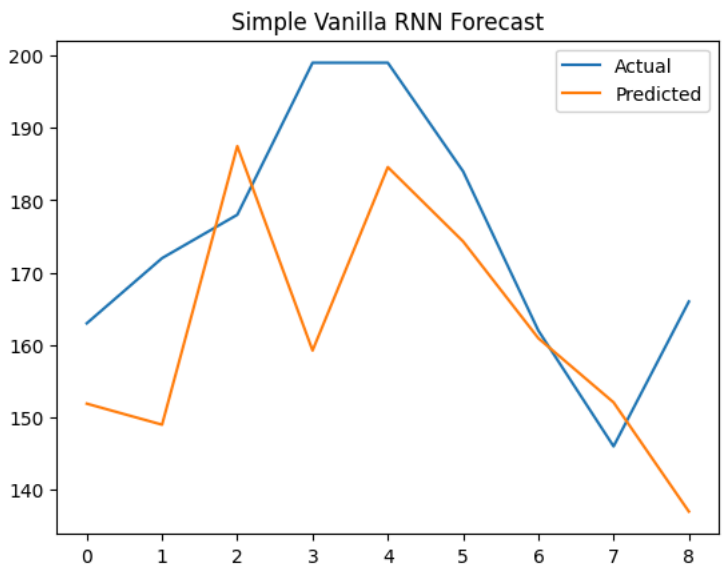
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argu
  super().__init__(**kwargs)
```

```
# 6. Train
history = model.fit(X_train, y_train, epochs=50, batch_size=10, verbose=1)
```

```
Epoch 1/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - loss: 0.0208
Epoch 2/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - loss: 0.0179
Epoch 3/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - loss: 0.0194
Epoch 4/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - loss: 0.0187
Epoch 5/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - loss: 0.0188
Epoch 6/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - loss: 0.0185
Epoch 7/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - loss: 0.0155
Epoch 8/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - loss: 0.0157
Epoch 9/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.0184
Epoch 10/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - loss: 0.0170
Epoch 11/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - loss: 0.0199
Epoch 12/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - loss: 0.0152
Epoch 13/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - loss: 0.0168
Epoch 14/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - loss: 0.0192
Epoch 15/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - loss: 0.0167
Epoch 16/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - loss: 0.0172
Epoch 17/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - loss: 0.0165
Epoch 18/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - loss: 0.0149
Epoch 19/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - loss: 0.0181
Epoch 20/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - loss: 0.0195
Epoch 21/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - loss: 0.0175
Epoch 22/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - loss: 0.0187
Epoch 23/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - loss: 0.0194
Epoch 24/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - loss: 0.0176
Epoch 25/50
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - loss: 0.0174
Epoch 26/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - loss: 0.0186
Epoch 27/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - loss: 0.0155
Epoch 28/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - loss: 0.0147
Epoch 29/50
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 46ms/step - loss: 0.0165
```

```
# 7. Predict
pred = model.predict(X_test)
pred_rescaled = scaler.inverse_transform(pred)
y_test_rescaled = scaler.inverse_transform(y_test.reshape(-1,1))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step
```

```
# 8. Plot results
plt.plot(y_test_rescaled, label="Actual")
plt.plot(pred_rescaled, label="Predicted")
plt.title("Simple Vanilla RNN Forecast")
plt.legend()
plt.show()
```